

PACE 2026

Problem

This year features the [Maximum-Agreement Forest \(MAF\)](#) problem arising in [phylogenetics](#) (the study of evolutionary histories).

See also the [Format Specification](#) and the [IPEC'25 Announcement Slides](#).

Timeline

- September 2025: Announcement of the challenge and tracks ✓
- October 2025: Definition of input and output formats ✓
- November 2025 (last week): **updated 04. Feb** Tiny test set ✓
- December 2025 (early): [Checker](#) ✓, [STRIDE tool](#) and [STRIDE database](#) ✓, [Rust library](#) ✓ for reading instance are provided
- ~~January~~ February 2026: Release of public instances and details about the benchmark ✓
- April 2026: Submission via [optil.io](#) opens
- July 2026: Final submission deadline and Results
- July 2026 (after submission): Review and rebuttal phase

Instances

- [Tiny example instances](#) (changes only to [summary.pdf](#))
- Public instances for **exact track**. [STRIDE list](#), [tar.gz](#)
- Public instances for **heuristic track**. [STRIDE list](#), [tar.gz](#)
- Public instances for **lower-bound track**. [STRIDE list](#), [tar.gz](#) **updated 03. March**
Due to a technical issue, we needed to slightly modify the lower bound instances. Please update.

Tools

The following software tools are available. There is **no** requirement to use them:

- [pace26stride](#) Solver runner and instance verifier with tight integration of the [STRIDE database](#) (see below).
- [pace26checker](#) (deprecated – use stride!): This tool implements a small verifier for instances and solutions.
- [pace26io](#): A software library implemented in Rust to parse input instances and write solution files.

STRIDE

STRIDE is a [tool](#) that has two main objectives:

- STRIDE offers a runner that allows you to execute and monitor your solver. You simply provide a set of instances and STRIDE will run your solver (in parallel), monitor its resource usage, verify the solution and provide a machine-readable summary.
- It is tightly integrated with our [STRIDE instance server](#). This server hosts several thousand instances together with solutions provided by the community. The majority of instances are small and should be relatively fast to solve; we recommend regularly testing your solvers against the database to detect errors and find regressions.

The server also accepts solutions, which are verified, stripped of any metadata, stored, and published after the competition is completed. In the meantime, we publish only the best known score for each instance. If you are using STRIDE instances, we kindly ask that you upload your solutions (this is the default behavior of the STRIDE runner when solving STRIDE instances).

Results

No results yet.

Submission Guideline

We ask all participants to publish their codebase and solver description on Github or a similar platform that is publicly available. The following files are required.

- A license file at the root containing the license
- An installation guide at the root that also contains information about external dependencies
- the codebase
- the solver description

The deadlines mentioned above apply. Note that changes on the codebase and the solver description after their respective deadlines are prohibited, except for small changes resulting from the reviewing phase in close contact with the organizers. We kindly ask all participants to send us an email by the submission deadline, including a link to their Github repository (or a similar platform). If your submission is eligible for the student ranking, please also include a brief note indicating this.

Docker Environment

A docker stack containing the evaluation environment is being prepared.

Tracks

The challenge features three distinct tracks:

1. an “exact” track, solving MAF on an arbitrary number of trees – for this track, the instances come with precomputed values of some parameters (along with proofs, e.g. decompositions of certain width)
2. a “heuristic” track, solving MAF on two trees – the instances will consist of two large trees and the allotted computation-time will be short
3. a “lower-bound” track, also with two trees per instance – the aim is to reach an approximate solution quickly

Scoring

Exact track

Every correctly solved instance awards 1 point. Whenever a solver does not finish an instance in time (time + grace limit exceeded), this instance awards 0 points. If a solver produces an infeasible or suboptimal answer on any instance, it will be disqualified. Solvers achieving the same score will be ranked according to their total runtimes.

Lower-bound track

Depending on speed, every correctly solved instance awards between 0.5 and 1 points. Whenever a solver does not finish an instance in time (time + grace limit exceeded), this instance awards 0 points. If a solver produces an infeasible solution or a solution violating its size constraint (see below) on any instance, it will be disqualified.

Suppose that a suitable solution was produced in t seconds in time (timeout of 10min + grace period of 10s). Then a score of $(2 - t/610)/2$ is awarded (i.e., 0.5 points for the solution and up to 0.5 points depending on speed).

In contrast to the exact track, suboptimal solutions are acceptable to a certain extend. Each instance contains an approximation line ($\#a$) with two parameters $1 \leq a < 1.5$ and $0 \leq b$. Let k^* be the (unknown) size of an MAF. Then, a solver needs to produce a solution of size at most $\lfloor ak^* \rfloor + b$; if a solver produces an infeasible solution or exceeds the instance-specific threshold, the solver will be disqualified.

Heuristic track

Depending on quality, every correctly solved instance awards between 0 and 1 points. Whenever a solver does not finish an instance in time (time + grace limit exceeded), this instance awards 0 points. If a solver produces an infeasible solution on any instance, it will be disqualified.

Fix an instance with n leaves, and let k^* be the best solution size known to the PC. Note that $k^* < n$ holds for every instance. Let k be the solution produced by a solver and let $u = \min(n, 2 \cdot k^*)$. The score for the instance is calculated according to the formula $f(k) = \left(\max \left(0, \frac{u-k}{u-k^*} \right) \right)^2$. This means that producing an optimal solution yields 1 point, producing a trivial solution or a solution outside factor 2 of the optimal solution yields

0 points, and improving an already good solution is more beneficial than improving a bad solution.

Overview

	Exact Track	Lower-Bound Track	Heuristic Track
Timeout	30min	10min	5min
Grace Time	10s	10s	10s
RAM	8 GB	8 GB	8 GB
Solution size	Optimal k^*	$\leq \lfloor ak^* \rfloor + b$ (see #a line)	$\leq n$
Instance score	1 per solved	$(2 - t/610)/2$ with t : runtime (in sec)	$(\max(0, [u - k]/[u - k^*]))^2$ with k^* : best known solution size, k : produced size, and $u = \min(n, 2k^*)$

Evaluation and correctness of exact and lower-bound solvers

This year we will again take special care to ensure the correctness of the exact and lower-bound solvers. We provide a large dataset of instances to test against, and highly recommend to [STRIDE](#) during development and before submission. Please make sure your solver treats corner cases, such as empty or edgeless intermediate trees correctly.

Shortly after submission, there will be a peer-review phase:

- We ask all teams to publish their source code (open source) and make it available to the other teams after submission (on Github or a similar platform). Shortly after the submission deadline for the solvers, we provide a list of links for all solvers on this webpage.
- With the submission we ask for a sketch of proof of correctness of the solver.
- Within two weeks after submission the teams may inspect the code of the other teams and point out mistakes or simply provide instances that yield wrong results for these solvers. This may either happen by contacting other teams directly (e.g., via GitHub issues or eMail) or by contacting the PC.
- In a following rebuttal phase of one week the teams may respond, and in close contact with the organizers fix small bugs in the code. Fundamentally wrong solvers will be disqualified. In particular, solvers that cannot guarantee the stated constraints for their respective track will be disqualified.
- We will also verify the solvers against a huge database of smaller random instances.
- The evaluation of the solvers will be based on a set of 100 private instances that are similar to 100 publicly available test instances.

Preliminary leaderboard on optil

As in previous years, a preliminary leaderboard on [optil](#) will be available.

Zulip

Join us on [Zulip](#) for discussions and updates.

Program Committee

- [Alexander Leonhardt](#) (Goethe-Universität Frankfurt)
- [Manuel Penschuck](#) (University of Southern Denmark, Odense)
- [Mathias Weller](#) (Université Gustave Eiffel, Paris)